

Chapter 13

BestComm

13.1 Overview

The following sections are contained in this document:

- [Section 13.2, *BestComm Functional Description*](#)
- [Section 13.12, *BestComm DMA Registers—MBAR+0x1200*](#)
- [Section 13.13, *On-Chip SRAM*](#)

BestComm provides an efficient, integrated approach to gathering and manipulating data sets from a broad range of communication interfaces. BestComm consists of:

- BestComm (based on the SmartDMA [SDMA] module), with interfaces to:
 - peripherals using the CommBus,
 - the processor using an IP bus,
 - the system main SDRAM via the processor bus interface (XLB),
- a defined set of communication-oriented peripherals,
- local buffer memory,
- standard bus interfaces.

The Direct Memory Access controller (DMA) module provides a flexible and efficient means to move blocks of data within the system. The DMA controller reduces the workload on the microprocessor, allowing it to continue execution of system software. The DMA microcode engine is tailored to efficiently transfer data across the internal bus architecture to memory and peripheral devices.

The DMA controller processes microcode tasks that are stored in local memory (SRAM 16 kBytes). A task is a sequence of instructions, referred to as *descriptors*, that specifies a series of data movements or manipulations. The DMA controller steps through the descriptors and executes the specified function in a similar fashion to a CPU executing a program.

For the MPC5200, BestComm consists of SDMA and the following peripheral interfaces:

- 10/100 Fast Ethernet Controller (FEC)
- I²C
- PCI
- ATA
- LocalPlus
- Peripheral Serial Controller (implementing a different mix of functionalities such as SPI, UART, CODEC 8-16-32 bits, AC97 controller, I2S, IrDA controller)

Many of the peripherals' port pins serve multiple functions, allowing flexibility in optimizing the system to meet a specific set of integration requirements. For a description of the pin multiplexing scheme and supported functions, refer to [Chapter 2, *Signal Descriptions*](#).

Other peripheral functions are included in MPC5200, but are not directly supported by BestComm. These peripherals include:

- A separate Serial Peripheral Interface (SPI), which:
 - supports a 6.25 MHz rate as a master
 - supports a 12.5 MHz rate as a slave
- USB Host/Hub controller
- MSCAN controller
- General Purposes Timers

13.2 BestComm Functional Description

The BestComm I/O subsystem consists of the following:

- a BestComm DMA Controller
- an on-chip 16 kBytes SRAM
- a set of peripheral interface modules with DMA controllable:
 - transmit (Tx)
 - receive (Rx)

The BestComm unit provides an interrupt control and data movement interface. The Interface is on a separate peripheral bus to several on-chip peripheral functions. This independent control of data movement leaves the G2_LE core free to concentrate on higher level activities, which increases overall system performance.

BestComm DMA can control data movement on the following peripherals and interfaces:

- PCI bus
- ATA Controller
- Ethernet
- PSC
- I²C
- IrDA
- LP bus interface

BestComm DMA performs general purpose DMA transfers. Most data transactions are between the peripheral/interface (typically a FIFO) and the system SDRAM.

BestComm allows up to 16 tasks to run simultaneously under the control of up to 32 DMA hardware requestors, user selectable from a possible 64 DMA request sources.

A hardware logic unit capable of basic logic operations (boolean arbitrary operations, shift, byte swap) plus some precoded CRC (CRC-16, CRC-CCITT, CRC-32, Internet Checksum) is also integrated in the SDMA engine.

BestComm uses internal buffers to prefetch reads and post writes such that bursting is used whenever possible. This optimizes both internal and external bus activity.

Speculative reads from system SDRAM may also be enabled to increase performance.

FIFO interfaces are implemented between the DMA and each peripheral/interface. As FIFOs are filled or emptied, automatic requests are made to the DMA unit. Based on programmable water mark levels (called ALARM and GRANULARITY level), the DMA unit moves data to and from the FIFOs. This method insures uninterrupted data movement at the given peripheral/interface rate.

13.3 Features summary

- A programmatic, deterministic capability for managing bus resources while servicing many data streams with individual latency and processing requirements.
- Single cycle access of peripheral and memory data.
- Support for up to 16 simultaneously enabled tasks (channels).
- Support for up to 32 separate DMA requestors at a time, user selectable from a possible 64 DMA request sources.
- Support for operations with up to 12 sources, or 11 sources and 1 destination.
- Simultaneous 32-bit reads and writes.
- Checksum generation.
- Endian conversion.
- Chaining/Scatter-gather capability.
- Support for packet-based I/O protocols (limitation might be dictated by performance when too much control is implemented within the task).

13.4 Descriptors

The DMA controller interprets a series of descriptors that specifies a sequence of data movements and manipulations. A collection of these descriptors is much like a program. The two types of descriptors are Loop Control Descriptors (LCDs) and Data Routing Descriptors (DRDs). These descriptors allow a “for”-loop programming style for the SDMA engine.

The LCDs specify the index variables (memory pointers, byte counters, etc.) along with the termination and increment values, while the DRDs specify the nature of the operation to perform.

13.5 Tasks

A task is a microcode program that embodies a desired function. An example could be to gather an ethernet frame, store it in memory and interrupt the processor when done. The multi-channel DMA supports sixteen simultaneously enabled tasks. By dynamically swapping task pointers in the task table, an unlimited number of tasks could be supported.

13.6 Memory Map/ Register Definitions

Memory organization is described in the register array pointed to by the Task Base Address Register (TaskBAR).

The TaskBAR identifies a location for a table of pointers to multi-channel DMA tasks (Task TABLE or Entry Table).

Each task has an entry (8 long words) that contains information about the microcode’s location (start address and stop address) in memory as well as pointers to the variable table to be used in the task, the Function Descriptor Table for the logic functions used within the task, the

Context Save area used during task switch/swap and some specific flags to enable performance affecting modes such as speculative reads, prefetch enable, readline and combined write.

A task's code should always be loaded into SRAM as the SDMA engine can fetch its descriptors from this internal memory with one cycle access per instruction. It is not recommended to place the code in SDRAM as there will then be a few overhead clocks which are needed to load the SDMA instruction unit.

13.7 Task Table (Entry Table)

The Task Table (or Entry Table) is a memory region containing pointers to each SDMA task. A Task Table Base Address Register (taskBAR) sets the location of the Task Table itself. Each entry in the Task Table contains pointers to the task's first descriptor, last descriptor, Variable Table, and other task-specific information.

13.8 Task Descriptor Table

Each Task Descriptor Table is a memory region containing the descriptors that comprise the task. The pointers in the Task Table define the beginning and end of each Task Descriptor Table.

13.9 Variable Table

Each task has a private 32-word Variable Table, where a word is four bytes (32 bits). According to the application requirements, the user initializes some of the words in the Variable Table as follows. The first 24 words are for pointers, counter values and initial data. The DMA Engine manipulates these variables as it executes loops. The next 8 words hold words-aligned, two-byte ("short word" or 16 bit word) increment variables.

13.10 Function Descriptor Table

An area of 256 bytes divided in 4 groups of 64 bytes. Each group can represent a set of 16 different Logic Functions belonging to a single execution unit. Every function is encoded with a single word (32 bits).

The implemented SDMA engine uses only one out of four potential Execution Units, execution unit 3, so all the functions needed by the task will be encoded in the third group (starting at offset 0xC0 from the start address of the Function Descriptor Table). The other words are reserved and must be written to '0' to maintain memory alignment.

For space optimization, tasks which use the same logic functions could share a single Function Descriptor Table avoiding the redundancy of re-writing the same table many times in SRAM.

13.11 Context Save Area

This is an area allocated for each task to allow the SDMA engine to save vital data (such as index values, etc.) during a task switch operation to allow later restoration.

The context save area should never be used or modified by the user as it is managed directly by the SDMA engine.

13.12 BestComm DMA Registers—MBAR+0x1200

A register overview is provided in [Section 3.2, Internal Register Memory Map](#).

Hyperlinks to the BestComm DMA registers are provided below:

- Section 13-1, *SDMA Task Bar Register* (0x1200)
- Section 13-2, *SDMA Current Pointer Register* (0x1204)
- Section 13-3, *SDMA End Pointer Register* (0x1208)
- Section 13-4, *SDMA Variable Pointer Register* (0x120C)
- Section 13-5, *SDMA Interrupt Vector, PTD Control Register* (0x1210)
- Section 13-6, *SDMA Interrupt Pending Register* (0x1214)
- Section 13-7, *SDMA Interrupt Mask Register* (0x1218)
- Section 13-8, *SDMA Task Control 0 Register* (0x121C)
- Section 13-9, *SDMA Task Control 2 Register* (0x1220)
- Section 13-10, *SDMA Task Control 4 Register* (0x1224)
- Section 13-11, *SDMA Task Control 6 Register* (0x1228)
- Section 13-12, *SDMA Task Control 8 Register* (0x122C)
- Section 13-13, *SDMA Task Control A Register* (0x1230)
- Section 13-14, *SDMA Task Control C Register* (0x1234)
- Section 13-15, *SDMA Task Control E Register* (0x1238)
- Section 13-16, *SDMA Initiator Priority 0 Register* (0x123C)
- Section 13-17, *SDMA Initiator Priority 4 Register* (0x1240)
- Section 13-18, *SDMA Initiator Priority 8 Register* (0x1244)
- Section 13-19, *SDMA Initiator Priority 12 Register* (0x1248)
- Section 13-20, *SDMA Initiator Priority 16 Register* (0x124C)
- Section 13-21, *SDMA Initiator Priority 20 Register* (0x1250)
- Section 13-22, *SDMA Initiator Priority 24 Register* (0x1254)
- Section 13-23, *SDMA Initiator Priority 28 Register* (0x1258)
- Section 13-24, *SDMA Request MuxControl* (0x125C)
- Section 13-26, *SDMA task Size 0/1* (0x1260)
- Section 13-26, *SDMA task Size 0/1* (0x1264)
- Section 13-30, *SDMA Debug Module Comparator 1, Value 1 Register* (0x1270)
- Section 13-31, *SDMA Debug Module Comparator 2, Value 2 Register* (0x1274)
- Section 13-31, *SDMA Debug Module Comparator 2, Value 2 Register* (0x1278)
- Section 13-36, *SDMA Debug Module Status Register* (0x127C)
- SDMA Reserved Register 3 (0x1280)

13.12.1 SDMA Task Bar Register—MBAR + 0x1200

sdma_task_bar_register

Table 13-1. SDMA Task Bar Register

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	taskBar																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
		16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb
R	taskBar																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Description
0:31	taskBar	TaskBAR is the pointer to the base address of the Task Table (Entry Table)

13.12.2 SDMA Current Pointer Register—MBAR + 0x1204

Table 13-2. SDMA Current Pointer Register

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	CurrentPointer																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31 lsb
R	CurrentPointer															
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:31	currentPointer	CurrentPointer contains the address of the currently executing DMA descriptor.

13.12.3 SDMA End Pointer Register—MBAR + 0x1208

Table 13-3. SDMA End Pointer Register

	msb 0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	EndPoint															
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31 lsb
R	EndPoint															
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:31	endPointer	EndPoint contains the address of the last descriptor in the currently executing SDMA task.

13.12.4 SDMA Variable Pointer Register—MBAR + 0x120C

Table 13-4. SDMA Variable Pointer Register

	msb 0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	VariablePointer															
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31 lsb
R	VariablePointer															
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:31	variablePointer	VariablePointer contains the starting address of the variable table for the currently executing task.

13.12.5 SDMA Interrupt Vector, PTD Control Register—MBAR + 0x1210

Table 13-5. SDMA Interrupt Vector, PTD Control Register

msb 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15																
R	Vector A[7:6]				INA[3:0]				Vector B[7:6]				INB[3:0]			
W																
RESET:	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1

16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 lsb																
R	T/I	TEA	HE	Reserved												PE
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:7	IntVect1	The Interrupt Vector register is used during interrupt acknowledge read cycles. The high order four bits are programmed by the user, and the low order four bits are decoded from either the current task number or execution unit. If any task interrupts are asserted, Interrupt Vector 1 is driven during the interrupt acknowledge cycle. If the task interrupts are negated and the execution unit interrupts are asserted, Interrupt Vector 2 is driven during the interrupt acknowledge cycle. The registers are set to the uninitialized vector \$0F by system reset. The interrupt A number is prioritized with IPR[15] the highest and IPR[0] the lowest. If all interrupt mask bits are set, then INA[3:0] = 1111 is read from this location. The interrupt B number is prioritized with the dbgInterrupt as the highest and euInterrupt[0] the lowest. If all interrupt mask bits are set, then INB[3:0] = 1111 is read from this location.
8:15	IntVect2	See above
16	T/I	T/I: Task/Initiator priority. Set to '1' to switch to "TASK priority" control; set to '0' to revert to INITIATOR (Requestor) Priority mode. The priority level of either the TASK or the initiator is set in the register IPR0 through IPR31
17	TEA	TEA: If set to '1' a TEA received by BestComm will be ignored and the task will NOT be halted. TEA indication can still trigger an interrupt if the proper mask bit is cleared in the Interrupt Mask Register and the TEA status bit plus the TASK number of the task which received the TEA are still updated in the Interrupt Pending Register.
18	HE	HE = 1; allows smartDMA higher task number same request priority to block current task, and allow arbitration. HE = 0; disables higher task number from blocking. This bit is cleared by reset.
19:30	—	Reserved
31	PE	Prefetch Disable: set to '1' to disable prefetch. Set to '0' to enable prefetch on CommBus

13.12.6 SDMA Interrupt Pending Register—MBAR + 0x1214

Table 13-6. SDMA Interrupt Pending Register

		msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	DBG	Rsvd			TEA	Etn[3:0]				EU[7:0]								
W																		
RESET:		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31 lsb
R	TASK[15:0]															
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0	DBG	Debug
1:2	—	Reserved
3	TEA	A TEA has been received by the currently running task. The corresponding task number is written in the Error Task Number field
4:7	Etn[3:0]	Error Task Number: when a TEA is received by the currently executing task its corresponding number is indicated here . If the TEA bit of the PtdControl register is set then the task will not be halted. If the TEA Msk bit in the Mask register is set then no interrupt to the core will be generated.
8:15	EU[7:0]	Execution Unit: only EU3 is valid for MPC5200
16:31	TASK[15:0]	Each bit corresponds to an interrupt source defined by the task number or execution unit. This register contains a registered copy of the interrupt signal that the interrupting source generates. The corresponding bit in the register reflects the state of the interrupt signal even if the corresponding mask bit is set. An interrupt is masked by setting the corresponding bit in the IntMask register. A bit is cleared by writing 1 to that bit location. Writing 0 has no effect. At system reset, all bits are initialized to logic 0. 0 = The corresponding interrupt source is not pending. 1 = The corresponding interrupt source is pending.

13.12.7 SDMA Interrupt Mask Register—MBAR + 0x1218

Table 13-7. SDMA Interrupt Mask Register

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	DBG	Reserved	TEA Msk	Reserved				EU[7:0]									
W																	
RESET:	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31 lsb
R	TASK[15:0]															
W																
RESET:	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Description
0	DBG	Debug: set to '1' to mask the "debug" interrupt (see the SDMA Debug Control Register)
1:2	—	Reserved
3	TEA Msk	TEA Mask: set to '1' to mask the TEA. If set to '1' and a TEA is received in the currently executing Task an interrupt is generated.
4:7	—	Reserved

Bit	Name	Description
8:15	EU[x]	Execution Unit: Only EU3 is present in MPC5200
16:31	TASK[15:0]	Each bit corresponds to an interrupt source defined by the task number or execution unit. An interrupt is masked by setting the corresponding bit. At system reset, all bits are initialized to logic 1. 0 = The corresponding interrupt source is not masked. 1 = The corresponding interrupt source is masked (no interrupt is generated).

13.12.8 SDMA Task Control 0 Register—MBAR + 0x121C

SDMA Task Control 1 Register—MBAR + 0x121E

Table 13-8. SDMA Task Control 0 Register

SDMA Task Control 1 Register

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	En	Val	Alw Init	IN[4:0]						Auto Start	High En	Hold	Rsvd	AS [3:0]			
W																	
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb
R	TCR1 (same as for TCR0)																
W																	
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0	EN	Each of the sixteen tasks has an associated task control register. Only one register is shown. At system reset, all bits are initialized to logic zeros. Enable - Task Enable 0 = Disabled 1 = Enabled This bit can be set or cleared by the programmer at any time when a task is enabled or disabled. This bit is also set by the PTD logic if the auto-restart bit is set and the task completes.
1	Val	Valid - Initiator Number is Valid 0 = Initiator is not valid 1 = Initiator is valid This bit is set by the engine logic when it obtains the requestor value from the first DRD that is parsed. This bit is cleared by the logic when the task completes. At system reset, this bit is cleared.
2	Alw Init	Always Init - Decode of the always initiator 0 = The always initiator is not being used 1 = The always initiator is being used This bit is a status bit only and is set and cleared by writing the initiator number into the Task Control Register.

Bit	Name	Description
3:7	IN[4:0]	InitNum[4:0] - Initiator number from task descriptor These bits are registered when the SDMA engine has parsed the first DRD to obtain the requestor number. These bits are cleared by system reset. These bits can be written by the programmer when the Hold Init Num bit is set or being set and the task is not enabled. At system reset, these bits are cleared.
8	Auto Start	Auto-Start - Task Start 0 = Task will not restart within program control 1 = Task will restart at end of task automatically. This bit can be set or cleared by the programmer at any time. This bit is also cleared if the SDMA engine encounters an error in the task. At system reset, this bit is cleared.
9	High En	High-Enable - High Priority Task Enable 0 = Normal task enable control 1 = High priority task enable control This bit can be set or cleared by the programmer at any time. This bit enables the SDMA to give priority to the enabled task function over running a task. At system reset, this bit is cleared.
10	Hold	Hold Init Num- Hold initiator number 0 = Allow the SDMA engine to update initiator number for task 1 = Keep current initiator number. This bit allows the initiator number to be set by the programmer and held for the complete task. The SDMA can not overwrite the programmed initiator except for the use of the always initiator which is contained in a separate control bit.
11	—	Reserved
12-15	AS[3:0]	ASNum[3:0] - Auto-Start Task Number These four bits contain the task number which will be auto-started when the Auto-Start control bit is set. At system reset, these bits are cleared.
16:31	TCR1	Task control register for task 1. Same bit layout as for TCR0

13.12.9 SDMA Task Control 2 Register—MBAR + 0x1220

SDMA Task Control 3 Register—MBAR + 0x1222

**Table 13-9. SDMA Task Control 2 Register
SDMA Task Control 3 Register**

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	TCR2																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb	
R	TCR3																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:15	TCR2	Task control register for task 2. Same bit layout as for TCR0
16:31	TCR3	Task control register for task 3. Same bit layout as for TCR0

13.12.10 SDMA Task Control 4 Register—MBAR + 0x1224

SDMA Task Control 5 Register—MBAR + 0x1226

Table 13-10. SDMA Task Control 4 Register

SDMA Task Control 5 Register

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	TCR4																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb	
R	TCR5																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:15	TCR4	Task control register for task 4. Same bit layout as for TCR0
16:31	TCR5	Task control register for task 5. Same bit layout as for TCR0

13.12.11 SDMA Task Control 6 Register—MBAR + 0x1228

SDMA Task Control 7 Register—MBAR + 0x122A

Table 13-11. SDMA Task Control 6 Register

SDMA Task Control 7 Register

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	TCR6																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb	
R	TCR7																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:15	TCR6	Task control register for task 6. Same bit layout as for TCR0
16:31	TCR7	Task control register for task 7. Same bit layout as for TCR0

13.12.12 SDMA Task Control 8 Register—MBAR + 0x122C

SDMA Task Control 9 Register—MBAR + 0x122E

**Table 13-12. SDMA Task Control 8 Register
SDMA Task Control 9 Register**

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	TCR8																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb	
R	TCR9																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:15	TCR8	Task control register for task 8. Same bit layout as for TCR0
16:31	TCR9	Task control register for task 9. Same bit layout as for TCR0

13.12.13 SDMA Task Control A Register—MBAR + 0x1230

SDMA Task Control B Register—MBAR + 0x1232

**Table 13-13. SDMA Task Control A Register
SDMA Task Control B Register**

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	TCRA																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb	
R	TCRB																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:15	TCRA	Task control register for task 10. Same bit layout as for TCR0
16:31	TCRB	Task control register for task 11. Same bit layout as for TCR0

13.12.14 SDMA Task Control C Register—MBAR + 0x1234

SDMA Task Control D Register—MBAR + 0x1236

**Table 13-14. SDMA Task Control C Register
SDMA Task Control D Register**

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	TCRC																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb	
R	TCRD																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:15	TCRC	Task control register for task 12. Same bit layout as for TCR0
16:31	TCRD	Task control register for task 13. Same bit layout as for TCR0

13.12.15 SDMA Task Control E Register—MBAR + 0x1238

SDMA Task Control F Register—MBAR + 0x123C

**Table 13-15. SDMA Task Control E Register
SDMA Task Control F Register**

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	TCRE																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb	
R	TCRF																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:15	TCRE	Task control register for task 14. Same bit layout as for TCR0
16:31	TCRF	Task control register for task 15. Same bit layout as for TCR0

13.12.16 SDMA Initiator Priority 0 Register—MBAR + 0x123C

SDMA Initiator Priority 1 Register—MBAR + 0x123D

SDMA Initiator Priority 2 Register—MBAR + 0x123E

SDMA Initiator Priority 3 Register—MBAR + 0x123F

Table 13-16. SDMA Initiator Priority 0 Register

SDMA Initiator Priority 1 Register

SDMA Initiator Priority 2 Register

SDMA Initiator Priority 3 Register

		msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	IPR0 Hold	Reserved					Prior [2:0]			IPR1								
W																		
RESET:		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

		16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb
R		IPR2									IPR3							
W																		
RESET:		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
Each of the thirty-two initiators has an associated priority level. Only one register is shown. All bits are set to '0 at reset.		
0	IPR0 Hold	Hold - Keep current priority of initiator 0 = Allow higher priority initiator to block current initiator 1 = Hold current initiator priority level This bit can be set or cleared by the programmer at any time. This bit allows the current initiator to hold priority until the initiator has negated or the task has finished. When this bit is cleared, an initiator with a higher priority will block the current initiator and force arbitration. At system reset, this bit is cleared.
1:4	—	Reserved
5:7	Prior[2:0]	InitPrior[2:0] - Initiator/Task priority level. These bits can be set by the programmer at any time. These bits control the priority of the requestor/task which will be serviced next depending on the setting of the T/I bit in the PtdControl register. The highest priority is level 7. The lower priority is level 0.. If more than one initiator/task contains the same priority then the order of the task within the Task table (task 7 highest to task 0 lowest) will set the priority.
8:15	IPR1	Initiator Priority register for initiator 1 (or Task1 if PtdControl[16]=1). Same bit layout as IPR0
16:23	IPR2	Initiator Priority register for initiator 2.(or Task2 if PtdControl[16]=1) Same bit layout as IPR0
24:31	IPR3	Initiator Priority register for initiator 3.(or Task3 if PtdControl[16]=1) Same bit layout as IPR0

13.12.17 SDMA Initiator Priority 4 Register—MBAR + 0x1240

SDMA Initiator Priority 5 Register—MBAR + 0x1241

SDMA Initiator Priority 6 Register—MBAR + 0x1242

SDMA Initiator Priority 7 Register—MBAR + 0x1243

Table 13-17. SDMA Initiator Priority 4 Register

SDMA Initiator Priority 5 Register

SDMA Initiator Priority 6 Register

SDMA Initiator Priority 7 Register

		msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R		IPR4								IPR5								
W																		
RESET:		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb
R		IPR6								IPR7								
W																		
RESET:		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:7	IPR4	Initiator Priority register for initiator 4 (or Task4 if PtdControl[16]=1) Same bit layout as IPR0
8:15	IPR5	Initiator Priority register for initiator 5 (or Task5 if PtdControl[16]=1) Same bit layout as IPR0
16:23	IPR6	Initiator Priority register for initiator 6 (or Task6 if PtdControl[16]=1) Same bit layout as IPR0
24:31	IPR7	Initiator Priority register for initiator 7 (or Task7 if PtdControl[16]=1) Same bit layout as IPR0

13.12.18 SDMA Initiator Priority 8 Register—MBAR + 0x1244

SDMA Initiator Priority 9 Register—MBAR + 0x1245

SDMA Initiator Priority 10 Register—MBAR + 0x1246

SDMA Initiator Priority 11 Register—MBAR + 0x1247

Table 13-18. SDMA Initiator Priority 8 Register

SDMA Initiator Priority 9 Register

SDMA Initiator Priority 10 Register

SDMA Initiator Priority 11 Register

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	IPR8									IPR9								
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31 lsb
R	IPR10								IPR11							
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:7	IPR8	Initiator Priority register for initiator 8 (or Task8 if PtdControl[16]=1) Same bit layout as IPR0
8:15	IPR9	Initiator Priority register for initiator 9 (or Task9 if PtdControl[16]=1) Same bit layout as IPR0
16:23	IPR10	Initiator Priority register for initiator 10 (or Task10 if PtdControl[16]=1) Same bit layout as IPR0
24:31	IPR11	Initiator Priority register for initiator 11 (or Task11 if PtdControl[16]=1) Same bit layout as IPR0

13.12.19 SDMA Initiator Priority 12 Register—MBAR + 0x1248

SDMA Initiator Priority 13 Register—MBAR + 0x1249

SDMA Initiator Priority 14 Register—MBAR + 0x124A

SDMA Initiator Priority 15 Register—MBAR + 0x124B

Table 13-19. SDMA Initiator Priority 12 Register
SDMA Initiator Priority 13 Register
SDMA Initiator Priority 14 Register
SDMA Initiator Priority 15 Register

		msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R		IPR12								IPR13								
W																		
RESET:		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31 lsb
R	IPR14								IPR15							
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:7	IPR12	Initiator Priority register for initiator 12 (or Task12 if PtdControl[16]=1) Same bit layout as IPR0
8:15	IPR13	Initiator Priority register for initiator 13 (or Task13 if PtdControl[16]=1) Same bit layout as IPR0

Bit	Name	Description
16:23	IPR14	Initiator Priority register for initiator 14 (or Task14 if PtdControl[16]=1) Same bit layout as IPR0
24:31	IPR15	Initiator Priority register for initiator 15 (or Task15 if PtdControl[16]=1) Same bit layout as IPR0

13.12.20 SDMA Initiator Priority 16 Register—MBAR + 0x124C

SDMA Initiator Priority 17 Register—MBAR + 0x124D

SDMA Initiator Priority 18 Register—MBAR + 0x124E

SDMA Initiator Priority 19 Register—MBAR + 0x124F

Table 13-20. SDMA Initiator Priority 16 Register

SDMA Initiator Priority 17 Register

SDMA Initiator Priority 18 Register

SDMA Initiator Priority 19 Register

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	IPR16								IPR17									
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb	
R	IPR18								IPR19									
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:7	IPR16	Initiator Priority register for initiator 16. Same bit layout as IPR0
8:15	IPR17	Initiator Priority register for initiator 17. Same bit layout as IPR0
16:23	IPR18	Initiator Priority register for initiator 18. Same bit layout as IPR0
24:31	IPR19	Initiator Priority register for initiator 19. Same bit layout as IPR0

13.12.21 SDMA Initiator Priority 20 Register—MBAR + 0x1250

SDMA Initiator Priority 21 Register—MBAR + 0x1251

SDMA Initiator Priority 22 Register—MBAR + 0x1252

SDMA Initiator Priority 23 Register—MBAR + 0x1253

Table 13-21. SDMA Initiator Priority 20 Register

SDMA Initiator Priority 21 Register

SDMA Initiator Priority 22 Register

SDMA Initiator Priority 23 Register

		msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	IPR20									IPR21								
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

		16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb
R	IPR22									IPR23								
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Description
0:7	IPR20	Initiator Priority register for initiator 20. Same bit layout as IPR0
8:15	IPR21	Initiator Priority register for initiator 21. Same bit layout as IPR0
16:23	IPR22	Initiator Priority register for initiator 22. Same bit layout as IPR0
24:31	IPR23	Initiator Priority register for initiator 23. Same bit layout as IPR0

13.12.22 SDMA Initiator Priority 24 Register—MBAR + 0x1254

SDMA Initiator Priority 25 Register—MBAR + 0x1255

SDMA Initiator Priority 26 Register—MBAR + 0x1256

SDMA Initiator Priority 27 Register—MBAR + 0x1257

Table 13-22. SDMA Initiator Priority 24 Register

SDMA Initiator Priority 25 Register

SDMA Initiator Priority 26 Register

SDMA Initiator Priority 27 Register

		msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R		IPR24								IPR25								
W																		
RESET:		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31 lsb
R	IPR26								IPR27							
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:7	IPR24	Initiator Priority register for initiator 24. Same bit layout as IPR0
8:15	IPR25	Initiator Priority register for initiator 25. Same bit layout as IPR0
16:23	IPR26	Initiator Priority register for initiator 26. Same bit layout as IPR0
24:31	IPR27	Initiator Priority register for initiator 27. Same bit layout as IPR0

13.12.23 SDMA Initiator Priority 28 Register—MBAR + 0x1258

SDMA Initiator Priority 29 Register—MBAR + 0x1259

SDMA Initiator Priority 30 Register—MBAR + 0x125A

SDMA Initiator Priority 31 Register—MBAR + 0x125B

**Table 13-23. SDMA Initiator Priority 28 Register
SDMA Initiator Priority 29 Register
SDMA Initiator Priority 30 Register
SDMA Initiator Priority 31 Register**

		msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R		IPR28								IPR29								
W																		
RESET:		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31 lsb
R	IPR30								IPR31							
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:7	IPR28	Initiator Priority register for initiator 28. Same bit layout as IPR0
8:15	IPR29	Initiator Priority register for initiator 29. Same bit layout as IPR0

Bit	Name	Description
16:23	IPR30	Initiator Priority register for initiator 30. Same bit layout as IPR0
24:31	IPR31	Initiator Priority register for initiator 31. Same bit layout as IPR0

13.12.24 SDMA Requestor MuxControl—MBAR + 0x125C

Table 13-24. SDMA Request MuxControl

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R		Req31		Req30		Req29		Req28		Req27		Req26		Req25		Req24		
W																		
RESET:		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb	
R		Req23		Req22		Req21		Req20		Req19		Req18		Req17		Req16		
W																		
RESET:		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:1	Req31	00: Requestor (RESERVED) 01-10: No active requestor 11: Always Requestor 31
2:3	Req30	00: Requestor (RESERVED) 01-10: No active requestor 11: Always Requestor 30
4:5	Req29	00: Requestor (RESERVED) 01-10: No active requestor 11: Always Requestor 29
6:7	Req28	00: Requestor (RESERVED) 01-10: No active requestor 11: Always Requestor 28
8:9	Req27	00: Requestor (RESERVED) 01-10: No active requestor 11: Always Requestor 27
10:11	Req26	00: Requestor IrDA TX (PSC_6) 01-10: No active requestor 11: Always Requestor 26
12:13	Req25	00: Requestor IrDA RX (PSC_6) 01-10: No active requestor 11: Always Requestor 25

Bit	Name	Description
14:15	Req24	00: Requestor I2C1_TX 01-10: No active requestor 11: Always Requestor 24
16:17	Req23	00: Requestor I2C1_RX 01-10: No active requestor 11: Always Requestor 23
18:19	Req22	00: Requestor I2C2_TX 01-10: No active requestor 11: Always Requestor 22
20:21	Req21	00: Requestor I2C2_RX 01-10: No active requestor 11: Always Requestor 21
22:23	Req20	00: Requestor PSC4_TX 01-10: No active requestor 11: Always Requestor 20
24:25	Req19	00: Requestor PSC4_RX 01-10: No active requestor 11: Always Requestor 19
26:27	Req18	00: Requestor PSC5_TX 01-10: No active requestor 11: Always Requestor 18
28:29	Req17	00: Requestor PSC5_RX 01-10: No active requestor 11: Always Requestor 17
30:31	Req16	00: Requestor LP 01-10: No active requestor 11: Always Requestor 16

The remaining 16 Requestors are fixed as follows:

Table 13-25. Fixed REquestors Table

REQUESTORS	Peripheral
REQ15	(RESERVED)
REQ14	PSC1_TX
REQ13	PSC1_RX
REQ12	PSC2_TX
REQ11	PSC2_RX
REQ10	PSC3_TX
REQ9	PSC3_RX
REQ8	PCI TX
REQ7	PCI RX
REQ6	ATA TX

Table 13-25. Fixed REquestors Table (continued)

REQUESTORS	Peripheral
REQ5	ATA RX
REQ4	FEC TX
REQ3	FEC RX
REQ2	(RESERVED)
REQ1	(RESERVED)
REQ0	ALWAYS

13.12.25 SDMA task Size0—MBAR + 0x1260

SDMA task Size 1—MBAR + 0x1264

Table 13-26. SDMA task Size 0/1

Bits	0,4,8,12, 16,20,24,28	1,5,9,13, 17,21,25,29	2,6,10,14, 18,22,26,30	3,7,11,15 19,23,27,31
R	srcSize[1]	srcSize[0]	dstSize[1]	dstSize[0]
W				

RESET:

At reset all Bits are set to 0

Bit	Name	Description
	srcSize[1:0]	Each of the 16 tasks can be programmed to use the source and destination sizes contained in one of the Task Size Registers. The task size information is used by the SDMA module to determine the source and destination transfer size of the operands. When the size contained the task descriptor is set to 2'b11 then the size field from the Task Size Control register is selected. srcSize[1:0] - source size 00 - Word (32 bit) 01 - Byte 10 - Word 11 - Word destSize[1:0] - destination size 00 - Word (32 bit) 01 - Byte 10 - Word 11 - Word

13.12.26 SDMA task 0 & task Size 1 map

Table 13-27. SDMA task Size Map

Offset	Register Name	Byte 0	Byte 1	Byte 2	Byte 3	Access
0x1260	task Size 0	TS[0:1]	TS[2:3]	TS[4:5]	TS[5:7]	R/W
0x1264	task Size 1	TS[8:9]	TS[10:11]	TS[12:13]	TS[14:15]	R/W

Bit	Name	Description
		See Table 13-26 for details. Each task has 4 bits allocated (2 for source and 2 for destination Size)

13.12.27 SDMA Reserved Register 1—MBAR + 0x1268

Table 13-28. SDMA Reserved Register 4

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	res1																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb	
R	res1																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:31	res1	Reserved

13.12.28 SDMA Reserved Register 2—MBAR + 0x126C

Table 13-29. SDMA Reserved Register 2

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	res2																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb	
R	res2																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:31	res2	Reserved

13.12.29 SDMA Debug Module Comparator 1, Value1 Register—MBAR + 0x1270

Table 13-30. SDMA Debug Module Comparator 1, Value1 Register

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
R	Value1																	
W																		
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31 lsb
R	Value1															
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:31	Value1	Debug Module Comparator 1 Value.

13.12.30 SDMA Debug Module Comparator 2, Value2 Register—MBAR + 0x1274

Table 13-31. SDMA Debug Module Comparator 2, Value2 Register

	msb 0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	Value2															
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31 lsb
R	Value2															
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:31	Value2	Debug Module Comparator 2 Value.

13.12.31 SDMA Debug Module Control Register—MBAR + 0x1278

Table 13-32. SDMA Debug Module Control Register

	msb 0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	Block Tasks															
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31 lsb
R	AA	B	Comparator Type 1	Comparator Type 2			and/ or	EU breakpoints					E	I	B	
W																
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:15	Block Tasks	Specify for each of tasks 15-0, whether to block that task with detection of a breakpoint (bit 0 halts TASK 15, bit 1 halts TASK 14, etc) 0 Do not block task 1 Block the task
16	AA	AutoArm—specifies whether or not the triggered bit dbgStatusReg[16] will be automatically reset to 0 following the saving of context for a breakpoint. This bit is set to 0 at reset. 0 Triggered bit will not be automatically reset 1 Triggered bit will be automatically reset
17	B	Breakpoint—This bit specifies whether or not to take a breakpoint. This bit is set to 0 at reset. 0 Disable breakpoints 1 Enable breakpoints
18:20	Comparator Type 1	Comparator 1 type—These bits specify the type of data that has been loaded into comparator 1; refer to Table 13-33 for the bit encoding.
21:23	Comparators Type 2	Comparator 2 type—These bits specify the type of data that has been loaded into comparator 2; refer to Table 13-34 for the bit encoding.
24	and / or	AND/OR—This specifies what type of operation is to be used with the comparators. This bit is set to 0 at reset. 0 Indicates an OR'ing of the comparators 1 Indicates an AND'ing of the comparators
25:28	EU breakpoints	euBreakpoint: These bits indicate that a breakpoint has occurred in one of the four execution units. Each execution unit has one bit dedicated to it. A 1 in any of these bits indicates that the associated execution unit has issued breakpoint. These bits are sticky and must be overwritten to continue. These bits are cleared to zero at reset. See Table 13-35 for the bit encoding. MPC5200 has integrated only EU3
29	E	Enable interrupt breakpoint. 0 Do not enable external breakpoint to cause a halt condition 1 Allow external breakpoint to cause a halt condition
30	I	Enable internal breakpoint 0 Do not enable internal breakpoint to cause a halt condition 1 Allow external breakpoint to cause a halt condition
31	EB	Master External Breakpoint (this bit must be always set to allow any kind of breakpoint to halt the task) 0 Disable external breakpoint 1 Enable external breakpoint

Table 13-33. Comparator 1 Type Bit Encoding

Encoding	Comparator Type 1
000	uninitialized
001	write address
010	read address
011	current pointer

Table 13-33. Comparator 1 Type Bit Encoding (continued)

Encoding	Comparator Type 1
100	task #
101	reserved
110	reserved
111	reserved

Table 13-34. Comparator 2 Type Bit Encoding

Encoding	Comparator Type 1
000	uninitialized
001	write address
010	read address
011	current pointer
100	task #
101	counter value
110	reserved
111	reserved

The reserved encodings are set to 0 indicating an uninitialized state.

Table 13-35. EU Breakpoint encoding

	EU3	EU2	EU1	EU0
Reset	0	0	0	0

It must be noted that even if a breakpoint is issued at a specific address the SDMA engine will halt ONLY at a “data aligned” boundary (for instance, if the task moves 32 bits of data per transaction and a breakpoint is set at address 0x02 then the task will be halted at offset 0x04).

13.12.32 SDMA Debug Module Status Register—MBAR + 0x127C

Table 13-36. SDMA Debug Module Status Register

	msb	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
R	Reserved												I	E	T		
W																	
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	lsb
R	dbgStatusReg[15:0]																
W																	
RESET:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Description
0:12	Reserved	Reserved
13	I	Interrupt—This bit indicates whether or not an interrupt has been taken. This bit is set to 0 at reset. It can be written by the user or the SDMA engine. 0 No Interrupt 1 Interrupt taken
14	E	External Breakpoint—This bit indicates detection of an external breakpoint. Status bit is sticky and requires a one (1) to be written to it to clear it. The writing of a zero (0) to this bit has no effect. This bit is set to zero (0) at reset. 0 No external breakpoint detected 1 External breakpoint detected
15	T	Triggered (dbgStatusReg[16])—This bit indicates that a SmartDMA breakpoint has occurred with the current settings. Status bit is sticky and requires a one (1) to be written to it to clear it. The writing of a zero (0) to this bit has no effect. This bit is set to zero (0) at reset. 0 Armed or normal operation 1 Triggered or debug mode
16:31	dbgStatusReg[15:0]	dbgTaskBlock (dbgStatusReg[15:0])—Each bit corresponds to one of the 16 task numbers. The value of the register bit reflects the debug state of the task number. A bit is cleared by writing a one to that bit location; writing a zero (0) has no effect. At system reset, all bits are initialized to logic zeros (0). 0 Unblocked or normal operation 1 Blocked, task has been blocked due to a breakpoint

13.13 On-Chip SRAM

MPC5200 contains 16KBytes of on-chip SRAM. This memory is directly accessible by the BestComm DMA unit. It is used primarily as storage for task table and buffer descriptors used by BestComm DMA to move peripheral data to and from SDRAM or other locations. These descriptors must be downloaded to the SRAM at boot.

This SRAM resides in the MPC5200 internal register space and is also accessible by the processor core. As such it can be used for other purposes, such as scratch pad storage. The 16kBytes SRAM starts at location MBAR + 0x8000.

13.14 Programming Model

The SDMA engine expects the programmer to initialize several things in memory including the Task Table and the Variable Table(s). These are described and illustrated in the following sub-sections. The various descriptors used in each task are also described below.

13.14.1 Task Table

The programmer must initialize the *taskBAR* register in the IPB Interface Module (offset 0x1200). The Task Table (sometime also referred to as Entry Table), whose format is shown in [Figure 13-1](#), should reside at the address specified by *taskBAR*.

The Task Table base address must be aligned to a 512-byte boundary. There are sixteen tasks, each of which has its own unique Task Descriptor Table (TDT) start pointer, TDT end pointer, Variable Table pointer, control information, and status information. The TDT start pointer is a 32-bit value that points to the first descriptor, an LCD, of that particular task. The remaining descriptors (LCDs and DRDs) should consecutively follow the first one in memory, except in special branching cases. The TDT end pointer is a 32-bit value that points to the last descriptor, which must be a DRD, of that particular task.

The 32-bit Variable Table pointer points to the top of the 32-word (128 byte) memory space where this task's Variable Table resides. The Variable Table format is explained later in more detail.

The control information is located in the fourth word of each task's Task Table information as shown in [Figure 13-1](#). Bits 0 through 23 contain the base address for this task's function descriptors. Control bits 24 through 31 are for precise increment, not resetting the error code, whether to pack data, integer mode, complex data mode, to enable speculative reads and whether bursting is allowed on reads and writes.

The fifth and sixth word of the task table are reserved.

The seventh word is a pointer to the Context Save Area where important data is saved and later restored in case of a task switch.

The last word is used by the SDMA engine in conjunction with Literal Initialization of LCD (to save variable usage). The user should not modify the values stored there.

	0		4	5		1	1		1	2		3						
						4	5		9	0		1						
Task 0	Task Descriptor Start Pointer																	
	Task Descriptor End Pointer																	
	Variable Table Pointer																	
	Function Descriptor Base Address								R	P	E	P	I	S	C	R		
								S	I				P	W	L			
								V					R					
	Reserved																	
	Reserved																	
Base Address for Context Save Space																		
	Literal Base 0	Reserved		Literal Base 1	Reserved													
.....																		
Task n	Task Descriptor Start Pointer																	
	Task Descriptor End Pointer																	
	Variable Table Pointer																	
	Function Descriptor Base Address										R	P	E	P	I	S	C	R
									S	I				P	W	L		
								V						R				
	Reserved																	
	Reserved																	
Base Address for Context Save Space																		

Note: For each task, the start pointer, end pointer, and variable table pointer are 32-bit values. For the task control bits, bits 0 through 23 are for the Function Descriptor Base Address, and bits 24 through 31 are RSV = Reserved, PI = Precise Increment, E = do not reset error code if '1', P = Pack data if '1', I = Integer mode if '1' (else fractional), SPR = speculative enable, CW = Combined Write Enable if '1', and RL = Read Line Buffer Enable if '1' speculative Reads if '1'

Figure 13-1. Task Table

Table 13-37. Behavior of Task Table Control Bits

Control Function	Value	Meaning
Precise Increment	0	Increments are allowed at any time the SDMA can do it
	1	Only increment at the end of an iteration
No Error Code Reset	0	Reserved
	1	Reserved
Pack	0	Do not pack data
	1	Pack data
Integer Mode	0	Fractional data representation
	1	Integer data representation
Speculative Reads	0	Disabled
	1	Enabled
Combined Write Enable	0	Do not enable combined writes
	1	Enable combined writes
Read Line Buffer Enable	0	Do not enable line reads
	1	Enable line reads

13.14.1.1 Integer Mode

This input signal is only valid if the pack signal is negated (set to '0'). This signal indicates if the SDMA engine should operate in integer mode or fractional mode. During integer mode, the engine sign-extends read data and the it reads the write size amount of data starting from the MSB position and drives it to the proper destination byte lanes as indicated by the write address.

During fractional mode, the engine zero-extends read data and the ADS reads the write size amount of data starting from the LSB position and drives it to the proper write byte lanes as indicated by the write address.

13.14.1.2 Pack

This input signal indicates that packing or unpacking of data should occur if the read size does not equal the write size. The pack signal has precedence over the integerMode signal.

This signal indicates to the SmartDMA that it should pack data when the source size does not match the destination size. When this signal is asserted, the SmartDMA should pack data, and the *integerMode* signal is ignored. Otherwise, the SmartDMA should not pack data. Packing data refers to the case where the SmartDMA will wait for a full word of data before passing the data to one of the memory interfaces.

13.14.2 Variable Table

Table 13-38 shows the Variable Table format to which each task must adhere. The Variable Table pointer that is located in the Task Table in Figure 13-1 points to the first location in this 32-word (128-byte) memory space.

If restoring, and the Variable Table has been modified, then the new Variable Table pointer is located at the end of the context save space for the corresponding task. The Variable Table for each task must be aligned to a 16-byte boundary (to aid in address calculation). Before executing a particular task, that task's Variable Table must be initialized with the appropriate data. Specifically, any constants, initial values, and increment values must be written to the Variable Table before executing the corresponding task.

Variables may be loaded into words 0 through 23. Increment values 0 through 7 may be loaded into words 24 through 31, respectively.

Any of the eight increment values may be used as normal Loop-Index Variables or Constants if they are not needed as increment values. All of these variables and increment values may be used to initialize loop-index registers. However, only variables 0 through 31 may be written by the ADS. At this time, if variables 24 through 31 are written, it is assumed that these variables should be treated as normal Loop-Index Variables or Constants and not as increment values. Also, note that Variable Tables may overlap if sharing the last eight variables with another task's Variable Table is desired. In addition, if a task does not use the last 16 variables, another Variable Table could start immediately after that task's increment values, so as to not waste memory.

Table 13-38. Variable Table per Task

#	Hex Offset	Contents	Comments
0	00	Loop-Index Variable or Constant 0	These twenty-four words (32 bits) are used for constant operands to the EUs, for initialization values, or for a place to write results straight to a variable in this table. These are typically preloaded by the CPU unless you are writing directly to a variable.
1	04	Loop-Index Variable or Constant 1	
2	08	Loop-Index Variable or Constant 2	
3	0c	Loop-Index Variable or Constant 3	
4	10	Loop-Index Variable or Constant 4	
5	14	Loop-Index Variable or Constant 5	
6	18	Loop-Index Variable or Constant 6	
7	1c	Loop-Index Variable or Constant 7	
8	20	Loop-Index Variable or Constant 8	
9	24	Loop-Index Variable or Constant 9	
10	28	Loop-Index Variable or Constant 10	
11	2c	Loop-Index Variable or Constant 11	
12	30	Loop-Index Variable or Constant 12	
13	34	Loop-Index Variable or Constant 13	
14	38	Loop-Index Variable or Constant 14	
15	3c	Loop-Index Variable or Constant 15	
16	40	Loop-Index Variable or Constant 16	
17	44	Loop-Index Variable or Constant 17	
18	48	Loop-Index Variable or Constant 18	
19	4c	Loop-Index Variable or Constant 19	
20	50	Loop-Index Variable or Constant 20	
21	54	Loop-Index Variable or Constant 21	
22	58	Loop-Index Variable or Constant 22	
23	5c	Loop-Index Variable or Constant 23	
24	60	Compare Type[31:29], Reserved[28:16], Increment Variable 0[15:0]	Variables 24 - 31 may be increment variables of the format shown to the left. Any of these variables may be used as normal Loop-Index Variables or Constants (like variables 0 - 23) instead.
25	64	Compare Type[31:29], Reserved[28:16], Increment Variable 1[15:0]	
26	68	Compare Type[31:29], Reserved[28:16], Increment Variable 2[15:0]	
27	6c	Compare Type[31:29], Reserved[28:16], Increment Variable 3[15:0]	
28	70	Compare Type[31:29], Reserved[28:16], Increment Variable 4[15:0]	
29	74	Compare Type[31:29], Reserved[28:16], Increment Variable 5[15:0]	
30	78	Compare Type[31:29], Reserved[28:16], Increment Variable 6[15:0]	
31	7c	Compare Type[31:29], Reserved[28:16], Increment Variable 7[15:0]	

When the user writes a program, or when the assembler converts the user's programs, the SDMA engine will use the initialization variables and constants that the user or processor should have loaded into the Variable Table. The initial index variables in the LCD tells the engine to allocate space for the resulting variables in the loop registers. The space will be allocated consecutively, so the user knows with which register each variable will be associated. This is important when the user's program tries to reference one of these previously allocated variables. Also, the eight increment variables in positions 24 through 31 of [Table 13-38](#) are preloaded by the processor, as programmed by the user.



Notes